

Teacher Meeting Scheduler & Attendance Management System

Product Requirements Document (PRD)

1. Overview

Purpose: A centralized platform for scheduling meetings with Google Meet, automating calendar management, attendance tracking, and reporting while eliminating manual processes.

Problem Solved: Organizations currently use disconnected tools (Google Calendar, Meet, Email, Excel) leading to:

- Manual scheduling conflicts
- Attendance visibility gaps
- Missing reminders
- Inconsistent reporting

This platform automates all of these through Google OAuth, Calendar/Meet integration, and analytics.

2. Key Objectives

- Schedule one-time and recurring meetings with automatic Google Meet link generation
- Synchronize meetings with participant calendars
- Send automated meeting invitations and reminders
- Track attendance and participation duration
- Generate downloadable attendance reports and analytics
- Provide centralized dashboard for all users

3. User Roles

Teacher (Meeting Organizer)

- Create, edit, and cancel meetings
- Invite candidates and manage availability
- View attendance reports and analytics
- Export reports in PDF/Excel

Candidate (Meeting Participant)

- View assigned meetings and join meetings
- Receive invitations and reminders
- View attendance history
- Cannot create, edit, or access other candidates' meetings

4. Authentication

Method: Google OAuth 2.0

Login Flow:

1. User clicks "Continue with Google"
2. Grant Google permissions
3. User account created automatically (first login)
4. Redirected to dashboard

User Data Stored: Google ID, Name, Email, Profile Image, Role, Last Login Timestamp

5. Meeting Management

Meeting Creation

Teachers can create one-time or recurring meetings with:

- Title, Description, Date, Start/End Times
- Candidate selection
- Meeting type (Interview, Technical Assessment, Training, Classroom, Mentorship, Mock Interview, Group Discussion)
- Optional notes

System Actions:

1. Create Google Calendar Event
2. Generate Google Meet Link automatically
3. Send meeting invitations to candidates
4. Add event to participant calendars

Rescheduling & Cancellation

- **Reschedule:** Event updated, participants notified, dashboard updated
- **Cancel:** Event removed, participants notified immediately

6. Notifications & Reminders

Meeting Invitation Email

- Meeting Title, Description, Date/Time
- Google Meet Link
- Organizer Name

Reminder Schedule

Reminder	Time
First	24 Hours Before
Second	1 Hour Before
Final	15 Minutes Before

Cancellation Notification

- Immediate notification sent to all participants

7. Dashboards

Teacher Dashboard

- Total/Upcoming/Completed/Cancelled Meetings
- Attendance Percentage
- Upcoming Meetings Widget
- Today's Meetings
- Attendance Analytics
- Calendar Overview

Candidate Dashboard

- Upcoming Meetings
- Meeting History
- Attendance Records
- Calendar View

8. Attendance Tracking

Data Captured:

Field	Description
Join Time	Actual join timestamp
Leave Time	Actual leave timestamp
Duration	Total time present
Status	Present/Late/Left Early/Absent

9. Reports & Analytics

Teacher Reports Include:

- Meeting Name, Candidate Name, Join/Leave Times, Duration, Attendance %
- Total meetings conducted, participants, attendance rate, no-show %, average duration

Export Formats: PDF, Excel (.xlsx)

Calendar Views: Monthly, Weekly, Daily

10. Availability Management

Teachers can configure:

- Working hours (e.g., 9 AM - 6 PM)
- Holidays and leave days
- Blocked time slots
- System prevents scheduling conflicts

11. Security & Audit

Security: Google OAuth 2.0, JWT Authentication, HTTPS, RBAC, Secure Cookies, API Validation

Audit Logs: User Login, Meeting Creation/Update/Cancellation, Attendance Updates, Report Downloads

Non-Functional Requirements:

- API Response Time: < 2 seconds
- Scalability: 10,000+ users, 100,000+ meetings
- Uptime: 99.9%

12. Technology Stack

Frontend: Next.js, React, TypeScript, Tailwind CSS, Redux Toolkit

Backend: Node.js, Express, TypeScript

Database: MongoDB (Users, Meetings, Attendance, Notifications, Audit Logs)

APIs: Google Calendar, Meet, Gmail

Services: Nodemailer/SendGrid (Email), Redis/BullMQ (Background Jobs)

Deployment: Docker, Nginx, Azure/AWS

13. Deliverables

- GitHub Repository (Source Code)
- Documentation (README, Setup Guide, Deployment Guide)
- API Documentation (Swagger/OpenAPI)
- Database Design (ER Diagram)
- Docker Configuration (Dockerfile, Docker Compose)

14. Acceptance Criteria

- Google OAuth login functional
- Teachers can create meetings with automatic Google Meet link generation
- Google Calendar events created automatically
- Meeting invitations sent and appear in participant calendars
- Automated reminders sent on schedule
- Attendance data captured and reports generated/exported
- Docker deployment functional
- Swagger documentation complete
- Production-ready code structure

15. Submission Method

Public Git Repository

Requirement: Submit all source code in a public GitHub repository

Repository Must Include:

- Clean commit history with meaningful messages
- .gitignore configured properly
- [README.md](#) with project overview
- All source code organized in proper folder structure
- Environment configuration templates (.env.example)

Access: Repository must be publicly accessible for evaluation

API Documentation

Format: Swagger/OpenAPI (preferred) or Postman Collection

Documentation Must Include:

- All API endpoints with HTTP methods
- Request/Response schemas
- Authentication requirements
- Error codes and status codes
- Query parameters and request body examples
- Rate limiting information (if applicable)
- Base URL and environment details

Delivery: API docs should be:

- Hosted in repository (e.g., `/docs` folder)
- Accessible via live Swagger UI (if deployed)
- Exported as JSON/YAML format

Instructions & Setup Steps

README Should Include:

1. **Project Overview** - Brief description and features
2. **Prerequisites** - Node.js version, npm/yarn, Docker, etc.
3. **Installation Steps:**
 - Clone repository
 - Install dependencies: `npm install`
 - Configure environment variables
 - Set up Google OAuth credentials
 - Database setup (MongoDB connection)
4. **Running the Application:**
 - Development mode: `npm run dev`
 - Production mode: `npm run build && npm start`
 - Using Docker: `docker-compose up`
5. **Running Tests** - Unit tests, integration tests (if any)
6. **API Endpoints** - Link to Swagger/API docs
7. **Deployment Guide:**
 - Docker deployment steps
 - Environment variables required
 - Cloud platform deployment (Azure/AWS)
8. **Folder Structure** - Explanation of directory organization
9. **Contributing** - Code standards and conventions
10. **Support/Contact** - Developer contact information